NAME: C. PRIYANKA

REG NO: 192511074

# 17) PROLOG PROGRAM TO SUM THE INTEGERS FROM 1 TO `N

## Aim

To write and execute a Prolog program to find the sum of integers from 1 to N using recursion.

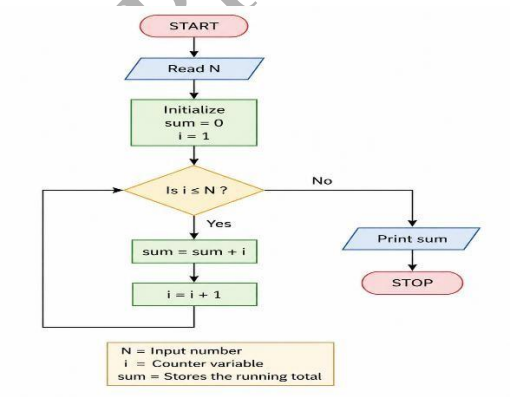
## Objectives

- To understand recursive programming in Prolog.
- To implement addition of integers from 1 to N.
- To learn the use of facts, rules, and queries in Prolog.
- To obtain the sum for a given value of N.

## Algorithm

1. Start.
2. Define the base case: the sum of integers from 1 to 0 is 0.
3. Define a recursive rule:
4. Sum from 1 to N = N + Sum from 1 to N-1.
5. Enter the value of N.
6. Recursively calculate the sum.
7. Display the result.
8. Stop.

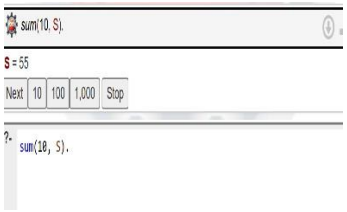
## Flow Chart



**Prolog Code**

```
% Sum of integers from
1 to N

sum(0, 0).

sum(N, S) :-
   N > 0,
   N1 is N - 1,
sum(N1, S1),
S is N + S1.
```

**Sample Output**




**Result**

Thus, the Prolog program successfully calculates the sum of integers from 1 to N using recursion.

**Conclusion**

The program demonstrates the use of **recursion and arithmetic operations in Prolog** to calculate the sum of a sequence of integers.

**18) PROLOG PROGRAM FOR A DATABASE WITH NAME AND DOB**

**Aim**

To write and execute a prolog program to create a database containing names and date of births and retrieve the information using queries.

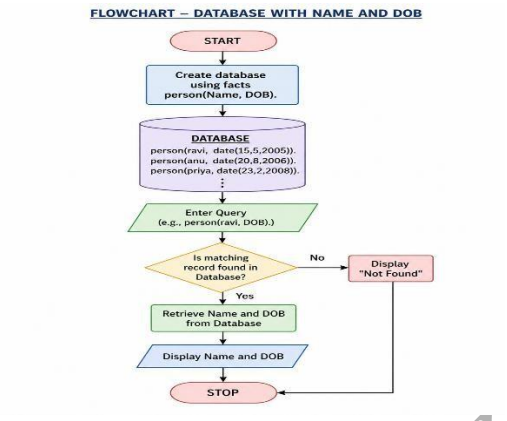
**Objectives**

- To understand database representation in Prolog.
- To store facts containing names and dates of birth.
- To retrieve information using Prolog queries.
- To understand the use of predicates and variables.

## Algorithm

1. Start.
2. Define a predicate person(Name, DOB).
3. Store names and their corresponding dates of birth as facts.
4. Load the Prolog program.
5. Enter a query to retrieve the DOB of a person.
6. Display the matching result.
7. Stop.

## Flow Chart




## Prolog Program

```
% Name and Date of Birth Database


person(ravi, date(15, 5, 2005)).

person(anu, date(20, 8, 2006)).

person(priya, date(23, 2, 2008)).

person(kiran, date(10, 12, 2005)).

person(rahul, date(7, 3, 2007)).
```

**Sample Output**


**Result**

Thus, the Prolog program successfully creates a **Name-DOB database** and retrieves the required information using queries.

**Conclusion**

The program demonstrates how facts and predicates can be used to represent and query a simple database in Prolog.

**19) PROLOG PROGRAM FOR STUDENT-TEACHER-SUBJECT CODE DATABASE**

**Aim**

To write and execute a Prolog program to create a database containing **student, teacher, and subject code information** and retrieve the required details using queries.

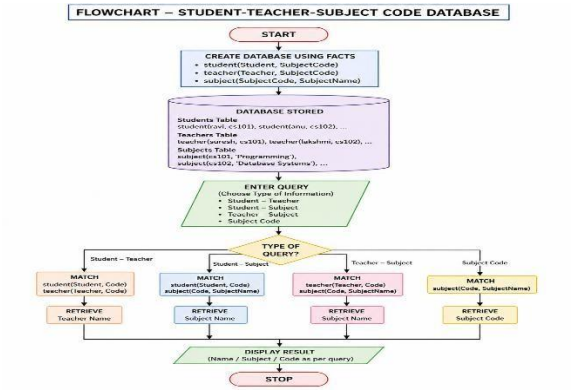
**Objectives**

- To create a relational database using Prolog.
- To represent student, teacher, and subject information.
- To establish relationships between students, teachers, and subjects.
- To retrieve information using logical queries.

**Algorithm**

1. Start.
2. Define a predicate teaches(Teacher, SubjectCode).
3. Define a predicate studies(Student, SubjectCode).
4. Store student-subject and teacher-subject relationships.
5. Enter a query.
6. Prolog searches the database for matching facts.
7. Display the result.
8. Stop.

## Flow Chart

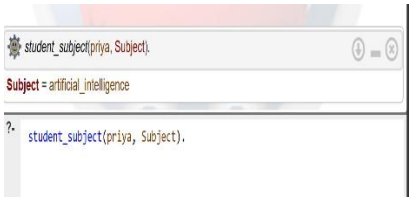



## Prolog Code

```
% Student-Teacher-Subject Code Database

student(ravi, cs101). student(anu,
cs102). student(priya, ai101).
student(kiran, cs101).
student(rahul, ai102).

teacher(suresh, cs101).
teacher(lakshmi, cs102).
teacher(rajesh, ai101). teacher(geetha,
ai102).

subject(cs101, programming). subject(cs102,
database).
subject(ai101, artificial_intelligence). subject(ai102,
machine_learning).

% Relationship between student and teacher
student_teacher(Student, Teacher) :-
student(Student, Code),     teacher(Teacher,
Code).

% Relationship between student and subject
student_subject(Student, Subject) :-
student(Student, Code),     subject(Code,
Subject).
```

% Relationship between teacher and subject teacher_subject(Teacher, Subject) :- teacher(Teacher, Code), subject(Code, Subject).

**Sample Output**




**Result**

Thus, the Prolog program successfully represents and retrieves **student, teacher, subject, and subject-code relationships**.

**Conclusion**

The program demonstrates how Prolog can be used to build a **relational database** and derive information through logical relationships and queries.

## 20) PROLOG PROGRAM FOR PLANETS DATABASE

**Aim**

To write and execute a Prolog program to create a database of planets and their properties and retrieve information using logical queries.

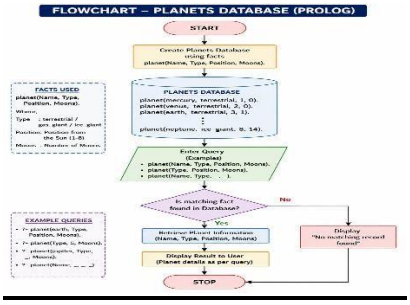
**Objectives**

- To create a simple planets database in Prolog.
- To store planet names and their properties.
- To retrieve planet information using queries.
- To understand database searching using Prolog predicates.

**Algorithm**

1. Start.
2. Define predicates for planet information.
3. Store the names of planets and their properties.
4. Load the Prolog program.
5. Enter a query for a particular planet.
6. Prolog searches the database.
7. Display the matching information.
8. Stop.

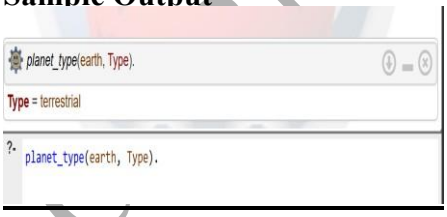
**FlowChart**




**Prolog Code**

```
% Planets Database

planet(mercury, terrestrial, 1).
planet(venus, terrestrial, 2).
planet(earth, terrestrial, 3).
planet(mars, terrestrial, 4).
planet(jupiter, gas_giant, 5).
planet(saturn, gas_giant, 6).
planet(uranus, ice_giant, 7).
planet(neptune, ice_giant, 8).

% Find planet type
planet_type(Planet, Type) :-
planet(Planet, Type, _).

% Find planet position from the Sun planet_position(Planet,
Position) :-
    planet(Planet, _, Position).
```

**Sample Output**




**Result**

Thus, the Prolog program successfully creates a Planets Database and retrieves planet information using Prolog queries.

**Conclusion**

The program demonstrates the use of facts, predicates, variables, and logical queries to represent and retrieve information from a planets database in Prolog.